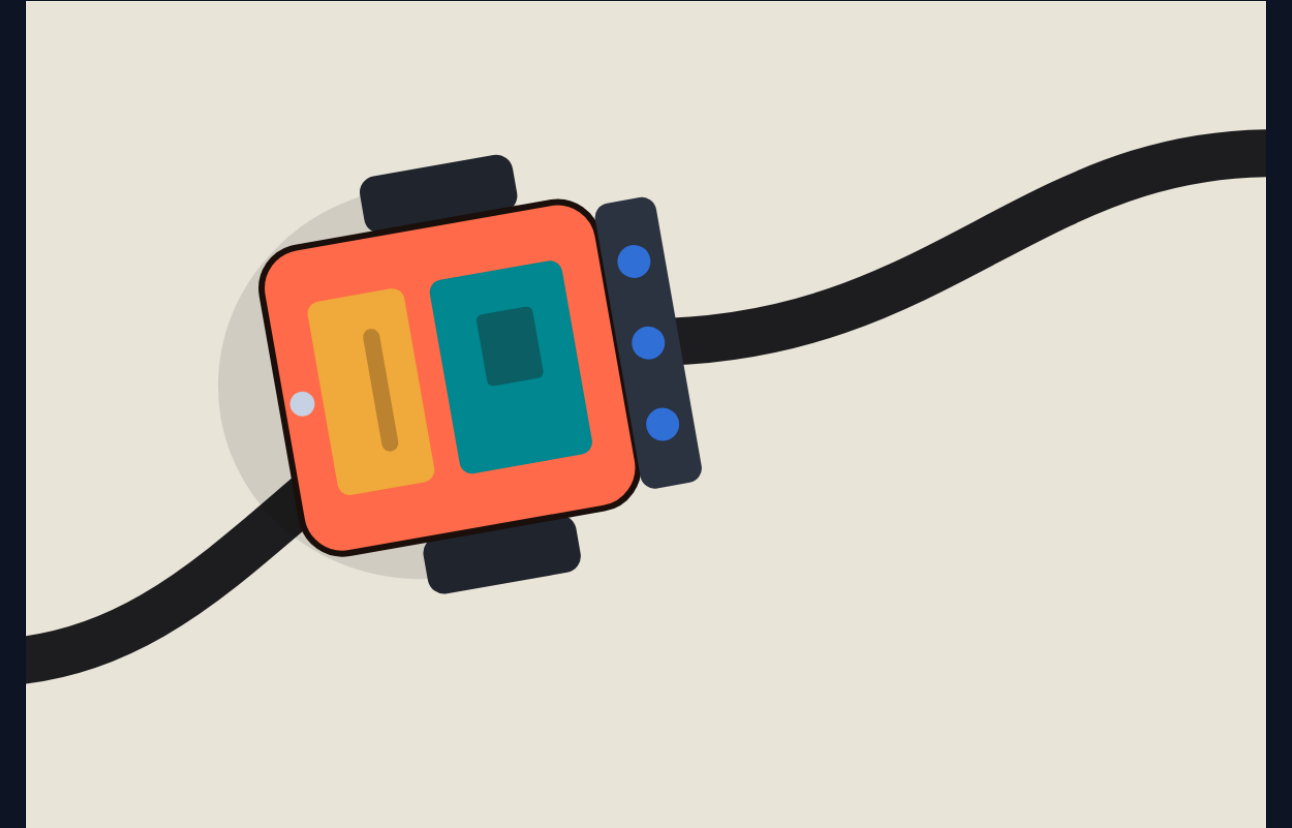


# Robot dün çalışıyordu

Bugün çizgiyi kaybediyor. Teslim yaklaşıyor.

Arif Solmaz  
İSTÜN Mekatronik Mühendisliği



---

Elif + Deniz · Bir robot · Ortak karar arayışı

# Hangi dosyaya güveneceğiz?

Elif başka, Deniz başka bir “son sürüm” gönderdi.

SORUN

Hangisi son hali?

23:47

Robot Projesi

.ino

robot.ino

geçmiş: her değişiklik, kim, ne zaman

Robot Takımı

K

Kaan

Elif, son kodu atar mısın?

E

Elif

robot\_final.ino'yu attım

robot\_final.ino

D

Deniz

Ama ben sensörü robot\_v2'de düzeltmiştim...

K

Kaan

Robot yine çizgiden çıktı. Hangisi son hali??

E

Elif

Bilmiyorum. Hepsini tek tek mi deneyelim?

GitHub'da tek bir **robot.ino** var; eski hâlleri geçmişinde saklı.

“SON” yazmak, geçmişini kaydetmek değildir.

# Dosya var. Karar nerede?

Takımın eksikliği kod miktarı değil, değişikliğin izi.

## Elimizde var

Kod dosyaları, birkaç kopya ve iki kişinin hatırladıkları.

## Bulmamız gerekiyor

Ne değişti? Neden değişti? Hangi hâlden devam edeceğiz?

---

İlk hedef: hatırlamak yerine karşılaştırmak.

## 01 · ROBOTUN İZİNİ BUL

# Dünle bugünü nasıl karşılaştırırız?

Geçmişte bir iz var: sensör eşiği 500 → 900.

2 FAYDA

Zaman makinesi: her değişiklik kayıtlı

E Elif'in ekranı

github.com/elif/robot-projem/commit/7f3e2a1

elif / robot-projem Public

<> Code Issues 0 Pull requests 0 Actions

Sensör eşiği değiştirildi

deniz committed yesterday 7f3e2a1

1 file changed

robot.ino +1 -1

3 int VIRAJ\_HIZI = 120; // virajda hız

4 - int ESIK = 500; // sensör eşiği

4 + int ESIK = 900; // sensör eşiği

5

6 void setup() {

<> Browse files

SENİN İÇİN

Proje için zaman makinesi

✓ Hiçbir değişiklik kaybolmaz.

✓ Kim, ne zaman, neyi değiştirdi: hepsi kayıtlı.

✓ Bozmaktan korkma: her kayıt bir geri dönüş noktası.

Commit = bir kayıt anı, yanında kısa bir not.

Buldun: eşik 500'den 900'e çıkmış. O günkü hâline dönüp bakabilirsin.

Değişikliği bulduk. Doğru eşiği henüz bilmiyoruz.

# Bu üç sayı neyi ayarlıyor?

GIF'teki sayılar örnektir. Gerçek eşik ölçümle seçilir.

```
// robot.ino – çizgi izleyen robot
int HIZ = 200;           // düz yolda hız
int VIRAJ_HIZI = 120;    // virajda hız
int ESIK = 900;          // sensör eşiği
```

| // yorum açar; **int** tam sayı değişkeni tanımlar.

| **ESIK = 900** sensör okumalarını karşılaştıran eşiktir.

| **HIZ = 200** düz yol, **VIRAJ\_HIZI = 120** viraj hız ayarıdır.

| Başlangıç değerini = verir. ; tanımı bitirir.

# Başlangıçta bu pinler ne yapıyor?

Bu parça pinleri hazırlar. Motoru süren devam kodu gösterilmiyor.

```
void setup() {  
  pinMode(5, OUTPUT); // sol motor  
  pinMode(6, OUTPUT); // sağ motor  
}
```

**setup()** her açılışta veya sıfırlamada bir kez çalışır. **void** değer döndürmez.

**pinMode(6, OUTPUT)** sağ motor sürücüsü sinyal pinini çıkış yapar.

**pinMode(5, OUTPUT)** sol motor sürücüsü sinyal pinini çıkış yapar.

**{ }** işlemlerin sınırındır; **//** sonrası açıklamadır.

# Bu işin hangi tarafı Git?

Geçmiş bilgisayarda tutulur. Ortak çalışma GitHub'da görünür olur.

## Git • yerel geçmiş

Değişiklikleri kaydeder. Hangi hâlden geldiğimizi gösterir.

## GitHub • ortak çalışma

Depoyu barındırır. İşleri, önerileri ve incelemeyi bir araya getirir.

---

Git, GitHub dışındaki Git sunucularıyla da kullanılabilir.

## 01 · ROBOTUN İZİNİ BUL

# Neyi birlikte saklayalım?

Robot kodu tek başına projeyi anlatmaya yetmiyor.

1 DAKİKA

Projeni GitHub'a koy

E Elif'in ekranı

github.com/elif/robot-projem

elif / robot-projem Public

<> Code Issues 0 Pull requests 0 Actions

main Add file <> Code

E elif Proje dosyaları eklendi 2 Commits

fotograflar Proje dosyaları eklendi

README.md Initial commit

devre-semasi.png Proje dosyaları eklendi

proje-raporu.pdf Proje dosyaları eklendi

robot.ino Proje dosyaları eklendi

README

**robot-projem**

Çizgi izleyen robot — ders projesi

SENİN İÇİN

Sadece kod değil: projenin her şeyi

✓ Arduino kodu

✓ Devre şeması

✓ Proje raporu

✓ Fotoğraflar

Repository (depo) = projenin internetteki klasörü.

Bitti: kod, devre şeması, rapor ve fotoğraflar artık internette.

Depo = proje dosyaları + kayıtlı geçmiş

# 0 satırda ne değişmiş?

500 → 900: aynı değişken, iki kayıt.

robot.ino

```
- const int ESIK = 500;  
+ const int ESIK = 900;
```

**robot.ino** Arduino kaynak dosyasının adıdır.

- önceki **500** satırını, + yeni **900** satırını gösterir.

**const int ESIK** bir tam sayı sabiti tanımlar.

= değer verir; ; tanımı bitirir; diff işaretleri programa yazılmaz.

# 900 yanlış mı?

Git değişikliği gösterir. Doğru eşiği robotta doğrularız.

- Geçmiş: ne değişti?
- Doğrulama: hangi koşulda nasıl davranıyor?
- Karar: hangi kanıta dayanarak kabul edeceğiz?

---

500 ve 900, bu kurgusal senaryonun örnek değerleridir.

## 01 · ROBOTUN İZİNİ BUL

# Bu sorunu kim takip ediyor?

İş kaydı: “IR sensör eşiğini incele.”

3 FAYDA

Takımın yapılacaklar listesi

E Elif'in ekranı

github.com/elif/robot-projem/issues

elif / robot-projem Public

<> Code Issues 2 Pull requests 0 Actions

Q is:issue state:open Labels New issue

2 Open 1 Closed Author Labels Assignee

Rapor için robotun fotoğrafını çek rapor #3 opened 2 minutes ago by elif

Motor sürücüsü ısınıyor bug #1 opened 2 days ago by kaan

SENİN İÇİN

Kim neyi yapıyor, herkes görür

İşler mesaj grubunda kaybolmaz; aylar sonra bile bulunur.

Her işin bir sorumlusu var.

Bitirme projesi, kulüp, ödev grubu: hepsine uyar.

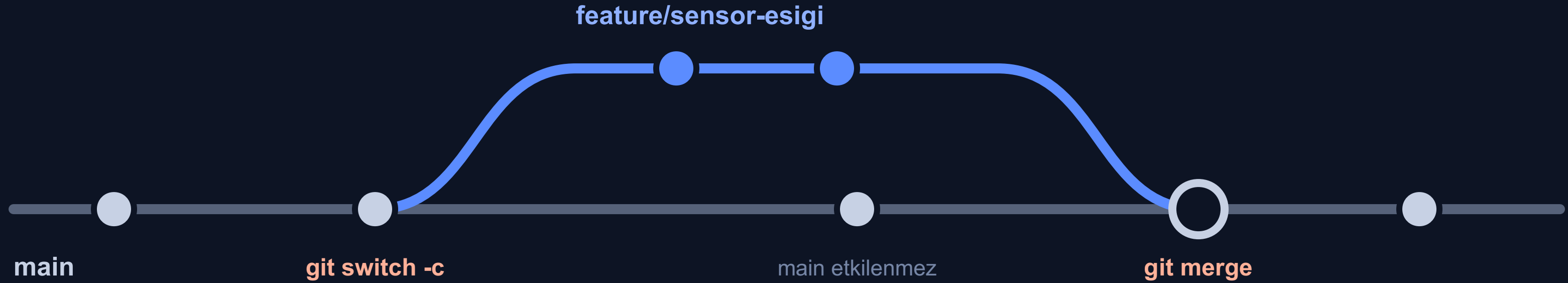
Issue = yapılacak bir iş ya da bir hata notu.

İş bitince **Close issue** : listeden düşer, kaydı kalır.

`is:issue state:open` = açık iş kayıtları; terminal komutu değildir.

# Main aynı kalırken nasıl deneriz?

Aday çözümü ayrı bir dalda geliştireceğiz.



**main** takımın ortak ana dalıdır.

**feature/sensor-esigi** sensör denemesinin dalıdır.

**git switch -c** dal açar; **git merge** çizgileri birleştirir.

Şemadaki **git switch -c** ve **git merge**, kısaltılmış komut etiketleridir.

# Dosyayı kaydettim. Deniz gördü mü?



**index** hazırlık alanı, **.git** yerel depo verileri, **HEAD** mevcut konumdur.

**origin** uzak adresin adı; **add**, **commit** ve **push** seçer, kaydeder, gönderir.

**pull** ve **clone** uzaktan alır; clone yeni yerel kopya başlatır.

# Aynı başlangıcı nasıl alırız?

Elif projeyi dosyaları ve geçmişiyle alıyor.

```
git clone <depo-adresi>  
cd robot-projem
```

**git clone** depoyu dosyaları ve geçmişiyle bilgisayara alır.

**cd robot-projem** terminali proje klasörüne geçirir; örnek adı oluşan klasör adıyla değiştir.

**<depo-adresi>** gerçek klonlama adresiyle değiştirilir.

# Deneme hangi dalda yapılacak?

Sensör denemesi için ayrı bir dal.

```
git switch -c feature/sensor-esigi  
git branch
```

| **git switch -c** mevcut konumdan yeni dal açıp ona geçer. | **feature/sensor-esigi** sensör işi için seçilen dal adıdır.

| **git branch** yerel dalları listeler; yıldız etkin dalı gösterir.

# Hangi çözümü deniyoruz?

900 → 500: henüz doğrulanmamış aday.

```
robot.ino
```

```
- const int ESIK = 900;  
+ const int ESIK = 500;
```

| **const int** tam sayı sabitini tanımlar; **;** tanımı bitirir.

| Diff işaretleri gerçek **robot.ino** dosyasına yazılmaz.

| - çıkarılan **900**, + eklenen **500** satırını gösterir.

# Bu kayda ne girecek?

Bu commit'e seçilen değişiklikler girecek.

```
git add robot.ino  
git diff --cached
```

**git add robot.ino** dosyanın o andaki hâlini hazırlar.

Add commit oluşturmaz veya göndermez; sonraki düzenleme yeniden add gerektirebilir.

**git diff --cached** hazırlanan hâli son commit ile karşılaştırır.

# Ne yaptığımızı nasıl kaydederiz?

Değişikliği yerel geçmişe kaydediyoruz.

```
git commit -m "IR sensör eşiği için 500 dönüş adayı"  
git log --oneline
```

**git commit** hazırlanmış hâli yerel geçmişe kaydeder.

**git log --oneline** kayıtları kısa kimlik ve mesajla listeler.

**-m** tırnak içindeki açıklamayı commit mesajı yapar.

# Deniz şimdi görebilir mi?

İş dalındaki kayıt artık takımda görünür.

```
git push -u origin feature/sensor-esigi
```

**git push** yerel commit'leri uzak depoya gönderir.

**-u** yerel dalın bu uzak dalı izlemesini kaydeder.

**origin** uzak adresin adı, **feature/sensor-esigi** gönderilen daldır.

Push iş dalını paylaşır; **main** kendiliğinden değişmez.

# Bu çözümü main'e alalım mı?

Adayı main'e almak için öneri açıyoruz.

```
feature/sensor-esigi → main
```

Sorun: Robot çizgiyi kaybediyor.

Aday: ESIK, 900 → 500.

Gerekçe: Önceki değere dönüşü incele.

Doğrulama: Ölçüm ve davranış kaydı bekleniyor.

**feature/sensor-esigi → main** önerinin kaynak ve hedef dalıdır.

Bu PR açıklaması terminal komutu değildir; merge yapmaz.

Sorun, aday, gerekçe ve doğrulama alanları incelemeyi yönlendirir.

# Deniz neye bakacak?

“Satır değişmiş” yeterli değil. Gerekçe ve davranış da anlaşılmalı.

- Neden bu değer?
- Hangi koşulda doğrulanacak?
- Başka bir davranışı etkiliyor mu?

---

Geri bildirim → aynı dalda yeni commit → güncellenen PR

# Aynı satıra iki karar gelirse?

İş dalında 500, gelen main'de 700.

```
robot.ino

<<<<<<< HEAD
const int ESIK = 500;
=====
const int ESIK = 700;
>>>>>>> main
```

| <<<<<<< **HEAD** mevcut iş dalının bölümünü açar.

| ===== tarafları ayırır; >>>>>>> **main** gelen bölümü kapatır.

| **500** iş dalındaki, **700** gelen **main** dalındaki aday eşiktir.

| Bu işaretler derlenebilir C++ değildir; seçilen çözümden kaldırılır.

# Hangisini seçiyoruz?

Bu örnekte 500 adayını sürdürüyoruz.

```
robot.ino  
  
const int ESIK = 500;
```

**const int ESIK** tam sayı sabitini tanımlar; **500** seçilen adaydır.

İşaretlerin kalkması doğrulama yerine geçmez; **500** evrensel eşik değildir.

Tek tanım kalır; <<<<<<, ===== ve >>>>>> kaldırılır.

# Karar ortak geçmişe nasıl girer?

İnceleme ve doğrulama yeterliyse öneri main'e alınır.

## Kabulden önce

İş dalında bir aday. Tartışılabilir ve değişebilir.

## Kabulden sonra

Ortak main'de bir karar. Gerekçesi projede bulunabilir.

---

Görünür olmak → incelenmek → kabul edilmek

# GitHub güncel. Bilgisayarım da mı?

Ortak main'i bilgisayara alıyoruz.

```
git status  
git switch main  
git pull --ff-only
```

| **git status** dosyaların ve dalın durumunu gösterir.

| **git pull --ff-only** uzak yeniliği yalnız ileri taşıma mümkünse alır.

| **git switch main** mevcut yerel ana dala geçer.

# Yeni biri robota nereden başlar?

README, projeyi devralana giriş yolu verir.

```
# Çizgi İzleyen Robot

## Donanım
Arduino, IR sensörler, motor sürücü.

## Başlarken
Bağlantı şemasını izle.
Sensör eşiğini kendi koşullarında doğrula.
Gözlemine ve değişiklik gerekçeni kaydet.
```

# ana başlık, ## bölüm başlığı oluşturur.

Donanım ve başlangıç satırları okuyucuya rehberlik eden düz metindir.

Bu Markdown bir **README.md** açıklamasıdır; terminal komutu değildir.

# Herkes görsün mü, herkes yazsın mı?

Görünürlük ile değiştirme yetkisi ayrı kararlardır.

## Görmek

Public: herkes görebilir. Private: erişim yetkiyle sınırlıdır.

## Değiştirmek

Ana depoya yazmak ayrıca izin gerektirir.

---

Parolalar ve erişim anahtarları proje dosyası değildir.

# Bulduğumuz kodu hemen kullanalım mı?

Bir örnek, ancak donanım ve kullanım koşullarıyla anlamlı.

- Donanım ve ortam uyuyor mu?
- Açıklama, örnek ve bilinen sorunlar var mı?
- Lisans neye izin veriyor?

---

Yıldız sayısı, robotumuzdaki doğruluğun ölçümü değildir.

# Görebildiğimiz kodu kullanabilir miyiz?

Lisans, paylaşımın koşullarını açıklar.

- Kullanım koşullarını oku.
- Gerekli bildirimleri koru.
- Kendi paylaşımının koşullarını açıkça belirt.

---

Public görünürlük, sınırsız kullanım hakkı anlamına gelmez.

# Dış projeye düzeltmeyi nasıl öneririz?

Kendi fork'umuza gönderir, asıl projeye PR açarız.

1. Katkı kılavuzunu oku.
2. Fork → clone → iş dalı → commit → push.
3. Kendi dalından asıl projeye PR öner.

---

Fork uzak kopyadır. Clone bilgisayardaki kopyadır.

## 04 · PROJEYİ DEVRET

# Bu proje bizi nasıl anlatır?

Dosyaları değil, yaptığımız işi anlaşılır hâle getiriyoruz.

5 FAYDA

GitHub profilin = portfolyon

github.com/elif/robot-projem

README.md

devre-semasi.png

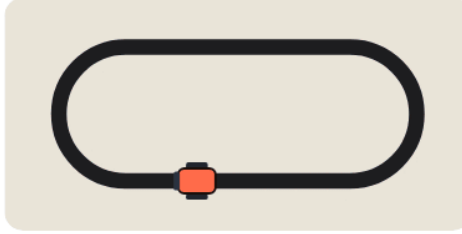
proje-raporu.pdf

robot.ino

README

**robot-projem**

Çizgi izleyen robot — Mekatronik dersi dönem projesi.



**Malzemele**

- Arduino U
- 2 DC mot
- 3 kızılöte
- 7.4 V Li-ion p

**Nasıl çalışır?**

Sensörler siyah çizgiyi görür. Robot çizgiden kaydıkaç motor hızlarını ayarlar ve yola geri döner.

Elif Aydın

Mekatronik Mühendisliği · 3. sınıf

PROJELER

[github.com/elif](https://github.com/elif)

SENİN İÇİN

Projelerin = portfolyon

✓

Hocalar ve işverenler ne yaptığını görür, sadece okumaz.

✓

Her proje bir vitrin: README'de fotoğraf, şema, açıklama.

✓

Staj ve iş başvurularında linkini CV'ne ekle.

README = projenin vitrini: ne, neden, nasıl.

Staj ya da iş başvurusu? CV'de tek satır yeter: [github.com/elif](https://github.com/elif)

Sorun · senin katkın · gerekçe · gösterim · bilinen sınırlar

# Dersten sonra nereden devam edilir?

İhtiyaç doğduğunda resmî kaynaklara geri dön.

- Kavramlar: Pro Git
- İşlem ayrıntıları: GitHub Docs
- Kendi hızında pratik: GitHub Skills

---

Kurulum ve komutlar: 15 başvuru slaytı • Öğrenci araçları: GitHub Education

# Takımın elinde şimdi ne var?

Artık hangi değişikliği neden paylaştığımızı anlatabiliyoruz.

## Başlarken

Farklı “son” dosyalar. Kaybolan gerekçeler. Kimin hangi hâlde olduğu belirsiz.

## Şimdi

Eşik kararının gerekçesi PR’da. Güncel main takımın bilgisayarlarında. README projeyi devretmeye hazır.

---

Git değişikliği kaydeder. Takım, gerekçesiyle karar verir.

# Yarın yine çizgiyi kaybederse?

Artık ilk sorumuz belli: “Ne değişti, hangi kanıtla?”

---

Değişiklik nerede? Kim görüyor? Neden kabul ediyoruz?

[GitHub Docs](#)

[Pro Git • Türkçe](#)

[GitHub Skills](#)

[GitHub Education](#)

# Git kurulumu

Terminal Git'i bulabiliyor mu?

```
git --version
```

**git --version** terminalin bulduğu Git sürümünü gösterir.

Git bulunmuyorsa resmî işletim sistemi kurulumuna bakılır.

Bu komut Git'i kurmaz veya depo oluşturmaz.

# Commit kimliği

Yeni commit'lerin adı ve e-postası.

```
git config --global user.name "Elif Örnek"  
git config --global user.email "elif@example.com"  
git config --global init.defaultBranch main
```

**--global** genel kullanıcı ayarıdır; depoya özel ayar öncelikli olabilir.

**init.defaultBranch main** yeni depolarda ilk dalı **main** seçer.

**user.name** committe görünen adı, **user.email** e-postayı belirler.

Bu bilgiler GitHub'a giriş veya gönderim yetkisi sağlamaz.

# Editör ve terminal

Aynı Git işlemi terminalde veya grafik arayüzde yapılabilir.

## Terminal

Komut ve seçenekleri açıkça görürsün; işlem adımları kolayca paylaşılır.

## VS Code

Dosyalar, fark görünümü, Source Control ve entegre terminal aynı pencerededir.

---

Open Folder ile proje açılır. View → Terminal ile terminal görünür.

# .git ve .gitignore

.git geçmiş, .gitignore izleme tercihini tutar.

```
robot-projem/  
  .git/  
  .gitignore  
  README.md  
  robot.ino  
  
# .gitignore örneği  
build/  
.env
```

**robot-projem/** proje klasörüdür; **.git/** yerel depo verilerini tutar.

**README.md** açıklama, **robot.ino** kaynak kod dosyasıdır.

**.gitignore** içindeki **build/** ve **.env** izlenmeyen yolları dışarıda bırakır.

**#** yorumdur; kurallar izlenen dosyayı veya eski geçmişini silmez.

# SSH anahtar çifti

Açık anahtar GitHub'a; özel anahtar kişide.

```
ssh-keygen -t ed25519 -C "elif@example.com"

# GitHub'a eklenecek dosya:
~/.ssh/id_ed25519.pub

# Paylaşılmayacak özel anahtar:
~/.ssh/id_ed25519
```

**ssh-keygen** anahtar çifti üretir; **-t ed25519** türünü seçer.

**.pub** açık anahtar GitHub'a eklenir; özel anahtar kişide kalır.

**-C** e-postayı yalnız yorum etiketi olarak ekler.

Bu dosya yolları komut değildir; mevcut anahtarın üzerine yazılmaz.

# SSH ajanı ve bağlantı

Bash / Git Bash için bağlantı örneği.

```
# Bash / Git Bash örneği
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# GitHub bağlantısını kontrol et
ssh -T git@github.com
```

**ssh-agent -s** ajanı başlatır; **eval** ayarları kabuğa uygular.

**ssh-add** özel anahtarı ajana yükler.

GitHub kabuk sunmaz; başarılı doğrulamada çıkış kodu **1** olur.

**\$(...)** içteki komutun çıktısını kullanır.

**ssh -T** bağlantıyı sınar; **-T** sanal terminal istemez.

# Yerel depoyu başlatma

Var olan klasörde depoyu başlat ve dosyaları seç.

```
cd robot-projem  
git init -b main  
git add README.md robot.ino
```

**cd robot-projem** terminali var olan klasöre geçirir.

**git add README.md robot.ino** iki dosyayı ilk kayıt için hazırlar.

**git init -b main** yerel depoyu ana dal adıyla başlatır.

# İlk commit ve uzak depo adresi

İlk commit yerelde, uzak adres origin adıyla bağlı.

```
git commit -m "Robot projesinin ilk kaydı"  
git remote add origin <depo-adresi>  
git remote -v
```

**git commit -m** hazırlanan hâli mesajla yerel geçmişe kaydeder.

**git remote -v** uzak getirme ve gönderme adreslerini listeler.

**git remote add origin** gerçek uzak adresi bu adla bağlar.

# İlk gönderim ve izleme

İlk gönderim uzak dalı izlemeye bağlar.

```
git push -u origin main

# İzleme bağlantısı kurulduktan sonra:
git push
git pull --ff-only
```

**git push -u origin main** gönderir ve dalın izleme bağlantısını kurar.

**git pull --ff-only** uzak yeniliği yalnız ileri taşıma ile alır.

Sonraki **git push**, olağan ayarlarda aynı hedefe gönderir.

**-u** seçeneğinin kurduğu izleme, fork'taki **upstream** uzak adıyla aynı şey değildir.

# İş dalı ve etkin dal

Yeni dal mevcut konumdan başlar.

```
git branch  
git switch -c feature/sensor-esigi
```

**git branch** yerel dalları ve yıldızla etkin dalı gösterir.

**feature/sensor-esigi** işin dal adıdır; mevcut konum başlangıçtır.

**git switch -c** yeni dal açar ve ona geçer.

# İki son hâl mi, ortak başlangıç mı?

Karşılaştırmanın başlangıç ve sonuç tarafını seç.

```
# İki dalın son hâli: main → feature
git diff main feature/sensor-esigi -- robot.ino

# Ortak başlangıçtan feature dalındaki değişiklik:
git diff main...feature/sensor-esigi
```

**git diff main feature/sensor-esigi** iki dalın son hâllerini karşılaştırır.

**main...feature/sensor-esigi** ortak başlangıç kaydından iş dalının sonuna bakar.

**-- robot.ino** görünümü yalnız bu dosyayla sınırlar.

Farkta - önceki, + sonraki karşılaştırma tarafıdır.

# Geri alırken niyeti seç

HEAD~1: önceki kayıt · <commit>: seçilen kayıt kimliği

## Hazırlıktan çıkar

```
git restore --staged README.md
```

Dosyayı hazırlıktan çıkarır. Dosyadaki düzenleme kalır.

## Paylaşılan değişikliği tersine çevir

```
git revert <commit>
```

Seçilen kaydın etkisini geri alan yeni commit oluşturur.

---

`git reset HEAD~1` dalı geri taşır; varsayılan `mixed` modu hazırlığı hedefe eşitler, çalışma dosyalarını korur.

# Hard reset ve kurtarma sınırları

Hard reset dosyaları hedef hâle getirir.

```
# Yalnız etkileri değerlendirilmişse:
```

```
git reset --hard <commit>
```

```
# Yerel işaretçi hareketlerini incele:
```

```
git reflog
```

**git reset --hard <commit>** dalı, hazırlığı ve izlenen dosyaları hedefe eşitler.

**git reflog** yerel işaretçi hareketlerinden eski commit'i buldurabilir.

Commit edilmemiş çalışma kaybolabilir; **<commit>** gerçek kimlikle değiştirilir.

Reflog sınırsız yedek değildir; hiç commit edilmemiş işi garanti etmez.

# Dosyayı seçme kaydetme ve gönderme

Komutları ihtiyaca göre kullan; tek betik değildir.

```
git clone <depo-adresi>
# Dosyayı düzenle ve kaydet
git status
git add <dosya>
git commit -m "Ne ve neden değişti?"
git push
```

**git clone** depoyu kopyalar; ardından proje klasörüne geçilir.

**git add <dosya>** seçilen dosyanın mevcut hâlini hazırlar.

**git push** yerel kayıtları uzağa gönderir.

**git status** dosyaların ve hazırlığın durumunu gösterir.

**git commit -m** hazırlanmış hâli mesajla kaydeder.

**<depo-adresi>** ve **<dosya>** gerçek değerlerle değiştirilir.

# Uzak güncelleme ve yeni iş dalı

Uzak yeniliği al; yeni işi ayrı dalda başlat.

```
# Takımın yeniliklerini al
git pull --ff-only
# İş dalı oluştur
git switch -c feature/<is>
```

**git pull --ff-only** yeniliği yalnız ileri taşıma mümkünse alır.

**<is>** gerçek iş adıdır; yeni dal düzenleme başlamadan açılır.

**git switch -c** mevcut konumdan yeni dal açıp ona geçer.